



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Laboratorio 11: Formularios Reactivos Avanzados y Consolidación Full-Stack

Ponderación: 5% de la nota final.

Modalidad: Individual.

Tiempo estimado de resolución: 4 horas.

1. Contexto del Negocio: ExpresoFast (Parte VII)

La plataforma logística **ExpresoFast** ha sido un éxito operativo gracias a la migración a una Single Page Application (SPA) con Angular Standalone en el laboratorio anterior. Sin embargo, el departamento de operaciones logísticas tiene un nuevo requerimiento crítico: **actualmente un envío solo representa una caja genérica, pero en la realidad, un mismo número de envío (tracking) puede contener múltiples paquetes (ítems) de distintos pesos y descripciones.**

Para solucionar esto, la gerencia ha solicitado construir una interfaz avanzada y dinámica que permita registrar un envío completo junto con todos sus paquetes asociados al mismo tiempo, validando las fechas operativas e impidiendo la duplicación de números de rastreo en tiempo real.

Como ingeniero Full-Stack de la empresa, usted debe diseñar esta solución manteniendo estricta adherencia a los estándares empresariales del curso: **SQL Server, Spring Boot 3 y Formularios Reactivos Tipados en Angular**.

2. Requerimientos del Back-End (SQL Server + Spring Boot)

Su aplicación Spring Boot (`expresofast-backend`) debe ser modificada para soportar esta nueva cardinalidad.

2.1 Actualización de la Base de Datos (SQL Server)

Se adjunta el script T-SQL oficial. Usted debe ejecutarlo en su base de datos local para crear la tabla de `PAQUETES` y establecer la relación `1:N` con la tabla existente de `ENVIOS`.

```
-- Ejecutar en SQL Server sobre la base de datos de ExpresoFast

CREATE TABLE PAQUETES (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
    envio_id BIGINT NOT NULL,
    descripcion VARCHAR(255) NOT NULL,
    peso_kg DECIMAL(5,2) NOT NULL,
    CONSTRAINT FK_Paquetes_Envios FOREIGN KEY (envio_id) REFERENCES ENVIOS(id) ON DELETE CASCADE
);
```

2.2 Mapeo JPA y Lógica de Negocio

1. **Nuevas Entidades:** Cree la entidad `Paquete` y configure la relación bidireccional `@OneToMany` / `@ManyToOne` con la entidad `Envio`.
2. **DTOs:** Actualice sus Data Transfer Objects (ej: `EnvioRegistroDTO`) para que al recibir el POST desde Angular, acepte una lista de `PaqueteDTO`.
3. **Transaccionalidad:** En su `EnvioService`, asegúrese de que la inserción del envío y de sus paquetes ocurra dentro del mismo contexto transaccional (`@Transactional`).
4. **Endpoint de Validación Asíncrona:** Cree un nuevo endpoint REST en su controlador: `GET /api/envios/check-tracking/{trackingNumber}` Este endpoint debe retornar un booleano o un objeto JSON indicando si el número de rastreo ya existe en la base de datos (para consumirlo desde el validador asíncrono de Angular).

3. Requerimientos del Front-End (Angular 19 + Reactive Forms)

Su aplicación Angular (`expresofast-frontend`) debe implementar un componente de formulario moderno y dinámico (`EnvioAvanzadoFormComponent`). Queda **estrictamente prohibido** el uso de Formularios Basados en Plantillas (Template-Driven Forms) mediante `[(ngModel)]` para la captura de estos datos.

Debe implementar los siguientes conceptos teóricos en código:

3.1 Formularios Estrictamente Tipados (Typed Forms)

Utilice `FormBuilder` (o `NonNullableFormBuilder`) para definir un `FormGroup` fuertemente tipado. TypeScript no debe permitir compilar si se intenta asignar un string al peso de un paquete.

3.2 Arreglos Dinámicos (FormArray)

El formulario debe contener una sección donde el operador pueda hacer clic en el botón **"+ Añadir Paquete"**. * Esta acción debe insertar dinámicamente un nuevo bloque de campos (`descripcion` y `pesoKg`) en un `FormArray`. * El usuario debe poder eliminar paquetes individuales con un botón **"X"** (siempre validando que exista al menos 1 paquete en el arreglo).

3.3 Validación Cruzada (Cross-Field Validation)

El formulario de envío captura dos fechas: `fechaDespacho` y `fechaEntregaEstimada`. * Desarrolle un **Validador Síncrono a nivel de FormGroup** que asegure matemáticamente que la `fechaEntregaEstimada` sea estrictamente mayor que la `fechaDespacho`. * Si la regla se incumple, debe mostrar un mensaje de error global y deshabilitar el botón de Submit.

3.4 Validación Asíncrona (Async Validator)

El campo `numeroTracking` es crítico y no puede repetirse. * Construya un **Validador Asíncrono (AsyncValidatorFn)** que, cada vez que el usuario escriba un número de rastreo, consulte al endpoint HTTP que usted construyó en el Back-End. * Si el número existe, el campo de texto debe

marcarse como inválido (`trackingTomado: true`) y desplegar el mensaje de error: "Este número de rastreo ya está en uso".

4. Requerimientos de Fundamentación Teórica

Para validar que usted domina las bases teóricas impartidas en los Temas 3 y 4 del curso, debe incluir un archivo `README.md` en la raíz de su repositorio (junto con su código fuente).

En este archivo, usted debe responder y fundamentar las siguientes dos interrogantes técnicas:

1. **UX y Escalabilidad:** Explique técnicamente por qué el uso de `FormArray` y formularios reactivos en Angular proporciona una mejor Experiencia de Usuario (UX) y una mayor mantenibilidad de código comparado con crear 10 campos de texto estáticos y ocultos en HTML.
2. **Ciclo de Eventos:** Con base en la arquitectura del Event Loop de JavaScript, explique la diferencia de ejecución entre su validador cruzado de fechas (síncrono) y su validador de tracking (asíncrono). ¿Por qué Angular requiere retornar un `Observable` o `Promise` en el validador asíncrono?

5. Entregables y Rúbrica de Evaluación (100 pts)

El código debe ser subido a un repositorio en GitHub, y el enlace debe ser enviado mediante Mediación Virtual. El repositorio debe contener ambas carpetas: `expresofast-backend` y `expresofast-frontend`, y el `README.md`.

Criterio Técnico	Ponderación	Nivel Excelente (100%)	Nivel Aceptable (70%)	Nivel Deficiente (0%)
------------------	-------------	------------------------	-----------------------	-----------------------

Arquitectura de Base de Datos y JPA	20 pts	Relación 1:N correctamente mapeada, guardando exitosamente en SQL Server de manera transaccional.	Entidades creadas pero con fallos en la cascada o transaccionalidad al guardar.	No se guarda la relación o se usa otra base de datos.
Typed Forms y FormArray	30 pts	Formulario estrictamente tipado, iterado correctamente en HTML. Permite agregar y remover paquetes sin errores de consola.	FormArray funciona parcialmente (errores al remover) o no utiliza tipeo estricto.	No usa FormArray o utiliza Template-Driven Forms.
Validación Cruzada Síncrona	20 pts	Validador de fechas aplicado al FormGroup, mostrando error visual en UI y previniendo el submit.	La validación funciona por debajo pero no se refleja al usuario, o se valida en el componente en el botón submit.	No implementa validación cruzada.

Validación Asíncrona (API)	20 pts	Endpoint API funcional, validador asíncrono retorna observable, consulta correctamente y bloquea duplicados.	Consulta a la API pero no enlaza correctamente el estado de error al FormControl.	No se implementa validación asíncrona contra la base de datos.
Fundamentación Teórica (README)	10 pts	Ambas preguntas teóricas contestadas con lenguaje técnico, profundo y sustentado.	Respuestas superficiales o sin sustento técnico.	No entrega README o respuestas incorrectas.

Nota sobre Plagio: Las defensas de código se realizan aleatoriamente. Si usted no puede explicar el funcionamiento de sus validadores o su interacción con el API en caso de ser seleccionado para la defensa presencial, perderá la totalidad de los puntos de este laboratorio.